

RAPPORT MÉTIER

Projet 12

Application agritech de prédiction de rendement et de recommandation de culture

1. Résumé exécutif

Le projet consiste à développer une application agritech capable de prédire le rendement agricole d'une culture et de recommander les cultures les plus intéressantes dans un contexte donné. La solution ne se limite pas à une simple estimation agronomique : les prédictions sont enrichies d'une couche économique permettant d'estimer le revenu, les coûts variables et la marge brute.

L'application associe un backend FastAPI, une interface Streamlit, plusieurs modèles de machine learning entraînés, une base SQLite modifiable pour les hypothèses économiques, ainsi qu'une chaîne d'industrialisation avec CI GitHub Actions, Docker et CD vers Docker Hub.

L'objectif métier n'est pas de produire une vérité agronomique absolue, mais un outil d'aide à la décision permettant d'explorer différents scénarios. L'utilisateur peut renseigner un pays, une surface, des données climatiques et des informations sur la fertilisation et l'irrigation. L'application calcule alors une prédiction de rendement, évalue les indicateurs économiques associés et propose, si demandé, un classement de cultures selon la marge brute estimée. Les hypothèses de prix et de coûts restant modifiables, l'outil est conçu comme un support d'exploration et de comparaison.

2. Contexte métier et besoin adressé

Dans un contexte agritech, une estimation de rendement seule est utile mais insuffisante pour prendre une décision opérationnelle. Une culture potentiellement productive n'est pas

nécessairement la plus intéressante si son prix de vente est faible, si les coûts variables sont élevés, ou si les hypothèses climatiques sont défavorables.

Le projet répond à un besoin plus large : croiser l'estimation agronomique avec une lecture économique simplifiée afin de proposer une aide à la décision plus proche d'un usage métier réel. Le besoin peut se formuler ainsi : pour une parcelle donnée, dans une zone géographique donnée, et sous certaines hypothèses techniques simples, quelle culture apparaît comme la plus intéressante — non seulement du point de vue du rendement, mais aussi de la marge brute potentielle ? Le projet apporte une première réponse en combinant une logique de prédiction, une logique de recommandation et une logique économique paramétrable.

3. Démarche analytique : des données à la modélisation

La construction de l'application n'a pas reposé sur un modèle unique entraîné directement sur un jeu de données brut. La démarche a consisté à structurer progressivement l'information disponible, à comparer plusieurs stratégies de modélisation, puis à retenir une architecture compatible à la fois avec les contraintes métier et avec la réalité des données.

3.1 Exploration des sources et stratégie de consolidation

Le projet s'appuie sur deux ensembles de données complémentaires. Le premier est un dataset principal de rendement agricole à grande échelle, contenant un volume important d'observations et des variables agronomiques et climatiques. Le second regroupe des sources plus hétérogènes, mobilisées pour reconstruire un dataset annexe consolidé à partir de données de rendement, de pluviométrie, de température et de pesticides.

L'exploration initiale a permis d'identifier six fichiers CSV dans l'environnement de travail. Parmi eux, quatre sources ont été retenues pour construire le dataset annexe utile à la recommandation. Cette étape a mis en évidence que les fichiers n'étaient pas directement alignés — ni sur les colonnes, ni sur la granularité, ni sur la qualité des clés de jointure.

Une phase d'harmonisation a donc été mise en place afin de : renommer les colonnes de manière cohérente, isoler les variables utiles, vérifier les recouvrements entre les clés géographiques et temporelles, et traiter les doublons — notamment sur la température, pour laquelle 7 994 clés Area-Year apparaissaient plusieurs fois avant agrégation.

Après traitement et fusion, le dataset annexe consolidé contient 13 136 lignes et 8 colonnes, sans doublon sur la clé métier (Area, Item, Year). Ce dataset constitue la base de travail du modèle secondaire de recommandation. Cette étape de consolidation est importante : la recommandation n'a pas été construite sur un rapprochement approximatif, mais sur un jeu de données explicitement nettoyé et restructuré.

3.2 Contrôle qualité et analyse exploratoire du dataset principal

Le dataset principal constitue le cœur de la modélisation de prédiction. Il contient initialement 1 000 000 de lignes et 10 colonnes. Le premier contrôle qualité a porté sur la variable cible — le rendement en tonnes par hectare — et a mis en évidence 231 valeurs négatives, avec un minimum observé de -1,1476 t/ha, incohérent au regard de l'objectif métier. Après suppression de ces valeurs, le dataset finalisé pour la modélisation contient 999 769 lignes.

Plusieurs constats importants se dégagent de cette phase d'exploration :

- le rendement moyen observé est de 4,65 t/ha ;
- la variable la plus fortement liée au rendement est la pluviométrie, avec une corrélation d'environ 0,76 ;
- la température présente une relation beaucoup plus faible avec la cible ;
- les moyennes de rendement par culture sont **très proches les unes des autres**.

Ce dernier point est central pour l'interprétation métier. Il signifie que, dans le dataset principal, la culture n'introduit pas à elle seule une différenciation marquée du rendement moyen, ce qui rend la discrimination entre cultures plus difficile pour le modèle. Ce constat analytique a directement influencé les choix de modélisation et la structure finale de l'application.

3.3 ACP et compréhension de la structure des données

Une analyse en composantes principales (ACP) a été conduite sur les deux jeux de données. Sur le dataset principal, elle a joué un rôle de lecture structurante : confirmer l'importance relative des facteurs climatiques et agronomiques, et objectiver la faiblesse de la séparation entre cultures. Sur le dataset secondaire, elle a évalué la pertinence d'une logique de profil par culture, renforçant l'idée que ce dataset pouvait compléter la couverture métier du système plutôt que remplacer le modèle principal.

L'ACP a ainsi rempli trois fonctions dans la démarche : confirmer les variables clés, objectiver la structure globale des données, et justifier les choix d'enrichissement et de routage entre modèles.

3.4 Construction des jeux enrichis et logique de profils

L'analyse exploratoire a conduit à construire plusieurs jeux de données dérivés. À partir du dataset secondaire consolidé, un travail de synthèse par culture a permis de produire des profils externes résumant les tendances agronomiques et climatiques observées. Ces profils ont ensuite servi à enrichir le dataset principal via l'ajout de variables supplémentaires : moyennes externes associées à une culture, écarts entre la situation observée et le profil moyen, et variables d'interaction destinées à renforcer la sensibilité du modèle à la culture considérée.

Un point important a toutefois émergé : les profils enrichis ne couvrent qu'un sous-ensemble exploitable de cultures, notamment Maize, Rice, Soybean et Wheat. Ce constat a directement justifié la mise en place d'une logique de routage différenciée entre modèles dans l'API.

3.5 Benchmark des modèles et sélection du modèle principal

La modélisation du moteur principal a suivi une démarche comparative structurée. Un benchmark léger sur échantillon a comparé plusieurs familles de modèles dans des conditions reproductibles. Les résultats obtenus sur la baseline générale sont les suivants :

- **Ridge** : RMSE $\approx 0,4976$, $R^2 \approx 0,9133$
- **CatBoost** : RMSE $\approx 0,4983$, $R^2 \approx 0,9131$
- **XGBoost** : RMSE $\approx 0,5006$, $R^2 \approx 0,9123$

Ces résultats montrent que le signal principal est déjà très bien capté par un modèle linéaire régularisé. Les modèles d'ensemble n'apportent pas de gain massif sur le cœur du dataset principal. Des variantes crop-aware ont ensuite été comparées (version enrichie, version avec ACP, version avec interactions) : les métriques sont restées très proches les unes des autres, confirmant que le problème tient moins au choix d'algorithme qu'à la faible différenciation intrinsèque entre cultures dans les données.

Sur le dataset complet, la baseline générale affiche des performances stables : RMSE $\approx 0,4993$, MAE $\approx 0,3983$, $R^2 \approx 0,9132$. La variante crop_enriched_interactions a finalement été retenue, car elle introduit une modulation légèrement plus sensible à la culture tout en restant compatible avec la logique applicative. Ce choix relève d'un compromis entre performance, robustesse, couverture fonctionnelle et utilité métier.

3.6 Interprétation métier du modèle principal

Le modèle prédit bien le rendement global avec un bon niveau de stabilité, mais n'introduit qu'une différenciation modérée entre cultures. Le contrôle métier réalisé dans le notebook montre que, dans un scénario donné, l'écart entre la culture prédite la plus haute et la plus basse reste de l'ordre de 0,017 t/ha. Cela signifie que le système capte bien l'effet des conditions générales sur le rendement, mais que la culture seule ne transforme pas fortement la prédiction dans le cadre du dataset principal.

Ce point justifie directement la suite de l'architecture : conserver un modèle principal performant pour la prédiction, un modèle général de repli, et compléter le dispositif par un modèle secondaire plus adapté à certaines recommandations ciblées.

3.7 Construction du modèle secondaire de recommandation

Le modèle secondaire a été construit à partir du dataset annexe consolidé. Après nettoyage et filtrage par support de cultures et de zones, le jeu final couvre 10 cultures et 90 zones géographiques (12 710 lignes). Cette réduction garantit que le modèle s'appuie sur des observations suffisamment représentées pour rester crédible en déploiement.

Le benchmark initial montre des performances nettement plus élevées sur ce périmètre spécifique :

- **Random Forest** : RMSE $\approx 1,365$, $R^2 \approx 0,969$
- **CatBoost** : RMSE $\approx 1,614$, $R^2 \approx 0,957$
- **HistGradientBoosting** : RMSE $\approx 2,212$, $R^2 \approx 0,919$
- **Ridge** : RMSE $\approx 4,206$, $R^2 \approx 0,706$

Ces résultats indiquent que le dataset secondaire est beaucoup plus favorable à une prédiction fine par culture et par zone. Le modèle déployable finalement retenu est un CatBoostRegressor optimisé (depth = 8, iterations = 800, l2_leaf_reg = 7, learning_rate = 0,1). La validation croisée du modèle final donne : RMSE moyen $\approx 2,177$, MAE moyen $\approx 1,200$, R^2 moyen $\approx 0,917$. Ces résultats sont compatibles avec un usage de recommandation. Le modèle secondaire n'a pas vocation à remplacer le modèle principal, mais à compléter le système lorsque la culture ou le contexte s'y prêtent mieux.

3.8 Conséquence directe sur l'architecture finale

Les résultats obtenus dans les notebooks expliquent directement la structure finale de l'application, reposant sur trois briques complémentaires :

- **model_1_crop** : lorsqu'une culture dispose d'un profil enrichi exploitable.
- **model_1_general** : solution de repli pour les cultures du dataset principal sans profil enrichi.
- **model_2_recommendation** : pour les cultures principalement couvertes par le dataset secondaire.

Cette architecture n'est pas un choix arbitraire d'implémentation. Elle résulte directement des observations faites lors de l'exploration des données et des benchmarks : le dataset principal est excellent pour la prédiction globale mais peu discriminant entre cultures ; le dataset secondaire est plus réduit mais mieux structuré pour certaines recommandations ciblées ; les profils externes sont utiles mais leur couverture est partielle. Le système traduit ainsi une logique pragmatique : exploiter le meilleur niveau d'information disponible selon la culture et le contexte, tout en conservant des garde-fous explicites sur la fiabilité des résultats.

3.9 Synthèse des enseignements analytiques

La phase de notebooks a permis d'aboutir à plusieurs enseignements structurants pour le projet :

1. Les données ne peuvent pas être exploitées efficacement sans un travail préalable de consolidation et d'harmonisation.
2. La pluviométrie constitue la variable la plus fortement liée au rendement dans le dataset principal.
3. La température joue un rôle plus secondaire que prévu dans la prédiction principale.

4. Les cultures présentent des rendements moyens relativement proches dans le dataset principal, ce qui limite la discrimination naturelle entre elles.
5. Les enrichissements externes apportent une modulation utile, mais partielle.
6. Le meilleur compromis applicatif ne repose pas sur un modèle unique, mais sur une architecture multi-modèles.
7. La recommandation gagne en pertinence lorsqu'elle est couplée à une lecture économique, et non réduite à un simple tri par rendement.

La phase d'exploration et de modélisation n'a pas seulement permis de sélectionner des algorithmes performants. Elle a surtout permis de construire une logique métier cohérente, dans laquelle les résultats analytiques expliquent directement la structure fonctionnelle de l'application finale.

4. Objectifs du projet

Trois objectifs fonctionnels structurent la solution :

- **Prédire le rendement d'une culture** : sous la forme d'un rendement par hectare, d'un rendement total sur la parcelle, d'une marge d'erreur et de bornes basses/hautes.
- **Recommander les cultures les plus intéressantes** : en classant plusieurs candidates selon la marge brute estimée.
- **Permettre la modification des données économiques** : depuis l'interface, afin de tester différents contextes de marché et de coûts.

Ces objectifs sont cohérents avec une logique de V1 métier. La solution n'ambitionne pas encore de couvrir toute la complexité économique du secteur agricole, mais elle fournit un socle applicatif complet : prédiction, recommandation, modification des hypothèses, exposition API, interface utilisateur et capacité de déploiement.

5. Périmètre fonctionnel de la V1

La V1 couvre deux cas d'usage principaux.

5.1 Prédiction : Cas mono-culture

L'utilisateur souhaite évaluer une culture précise. L'API /predict retourne une estimation de rendement, un intervalle de confiance simplifié, les variables effectivement utilisées, le modèle mobilisé, ainsi que les indicateurs économiques lorsque des prix et des coûts actifs sont disponibles. Les schémas Pydantic formalisent explicitement cette structure de réponse.

5.2 Recommandations : Cas multi-cultures

L'utilisateur souhaite obtenir un classement de cultures pertinentes. L'API /recommand retourne une liste structurée d'objets contenant la culture, le rendement prédit, les indicateurs économiques, un score de recommandation normalisé entre 0 et 1, un niveau de confiance, la source du modèle utilisé et une justification textuelle. Le résultat inclut également la liste des cultures écartées faute de données économiques actives.

6. Architecture générale de la solution

L'architecture repose sur une séparation claire entre interface, API, chargement des artefacts, services métier et couche économique. Le projet s'organise autour de cinq blocs principaux :

- **app/** — API FastAPI et logique métier
- **ui/** — interface Streamlit
- **scripts/** — initialisation de la base économique
- **artifacts/** — modèles et métadonnées
- **tests/** — tests API

L'API FastAPI expose quatre routes : / pour un message d'accueil, /health pour l'état des modèles, /predict pour la prédiction unitaire et /recommend pour la recommandation.

L'interface Streamlit se connecte à l'API via des appels HTTP, affiche l'état de santé du backend, collecte les entrées utilisateur et expose les résultats. Un troisième onglet est dédié aux données économiques modifiables.

7. Chaîne de traitement métier

Le fonctionnement global s'articule en plusieurs étapes. L'utilisateur renseigne une zone géographique, une surface, éventuellement des données climatiques ainsi que des informations sur la fertilisation et l'irrigation. Ces informations sont transmises à l'API sous forme de payload JSON.

Avant toute prédiction, les schémas Pydantic valident la cohérence des entrées : la surface doit être strictement positive, la pluviométrie ne peut pas être négative, top_k est borné entre 1 et 20, et les champs texte sont normalisés.

La couche services.py applique ensuite la logique métier centrale. Elle normalise les noms de culture et de zone, puis tente un autoremplissage climatique à partir d'un référentiel géographique dérivé des métadonnées du modèle 2. Si l'utilisateur n'a pas fourni les valeurs de pluviométrie ou de température, celles-ci peuvent être injectées à partir des moyennes du pays lorsque celui-ci est connu et supporté. Le système mémorise si les valeurs climatiques proviennent de l'utilisateur ou de l'autoremplissage.

Après cette étape, l'API choisit le modèle de prédiction le plus pertinent, produit une estimation de rendement, calcule des bornes à partir des métadonnées du modèle, puis

appelle la couche économique pour estimer le revenu, les coûts variables et la marge brute. La réponse finale intègre un niveau de confiance et des avertissements éventuels — notamment en cas de recours au modèle général, à l'autoremplissage climatique ou à une faible représentation de la culture dans les données d'entraînement.

8. Logique de modélisation et de routage

La solution ne repose pas sur un modèle unique. Au démarrage, l'application tente de charger trois artefacts principaux : `model_1_general`, `model_1_crop` et `model_2_recommendation`, ainsi que leurs métadonnées JSON. Elle cherche également un fichier `crop_profile.csv` pour reconstruire certaines features enrichies du modèle principal culture-spécifique.

La logique de routage dans `predict_crop_yield` suit trois niveaux de priorité :

- Si la culture dispose d'un profil enrichi, la prédiction est faite avec **`model_1_crop`**.
- Si la culture appartient au dataset principal sans profil enrichi, l'application bascule vers **`model_1_general`**.
- Si la culture n'est disponible que dans le modèle secondaire, la prédiction repose sur **`model_2_recommendation`**, à condition que l'aire géographique soit supportée.

Cette logique évite de fragiliser l'application lorsqu'une culture n'est pas couverte de façon homogène. Elle permet aussi de distinguer plusieurs niveaux de fiabilité — high, medium ou low — selon le modèle utilisé, le support de la culture et le recours éventuel à l'autoremplissage climatique.

9. Logique de recommandation multi-cultures

La recommandation est plus sophistiquée qu'un simple tri par rendement. Le service `recommend_crops_service` construit une liste de candidats sur trois niveaux :

- **Tier 1** : cultures disposant d'un profil enrichi, évaluées via `model_1_crop`.
- **Tier 2** : cultures du dataset principal sans profil enrichi, évaluées via `model_1_general`.
- **Tier 3** : cultures couvertes uniquement par `model_2_recommendation`, selon compatibilité géographique.

Pour les cultures du tier 1, un second avis peut être calculé via `model_2_recommendation` lorsque l'aire est compatible. Un coefficient d'ajustement de cohérence (`compute_consistency_adjustment`) module légèrement le score selon l'écart relatif entre les deux prédictions, renforçant ou nuancant la recommandation principale.

Le classement final ne repose pas uniquement sur la marge brute. Les candidats sont normalisés via `normalize_recommendation_scores`, puis triés par niveau de priorité métier (tier), par score de recommandation décroissant, puis par marge brute décroissante, avant

d'être tronqués à top_k. Cette hiérarchisation reflète une vraie logique produit : les cultures les mieux couvertes sont privilégiées, avant les cas de repli et les cas exploratoires.

10. Couche économique et valeur métier

La couche économique constitue un apport central du projet. Deux tables SQLite structurent la V1 : crop_prices pour les prix de vente et crop_costs pour les coûts variables par hectare. Chaque table intègre des colonnes métier utiles : valeur, unité, devise, source, date d'observation, statut par défaut, surcharge utilisateur, activation et date de mise à jour — offrant un premier niveau de traçabilité et de flexibilité.

L'estimation économique se fait dans compute_economic_outputs. Le système convertit si nécessaire un prix exprimé en usd_per_kg en usd_per_tonne, calcule le rendement total à partir de la surface, estime le revenu attendu, puis déduit les coûts variables actifs pour produire une marge brute estimée. Si aucun prix actif n'existe pour une culture, la fonction renvoie des sorties économiques à None, ce qui empêche qu'une culture soit recommandée sans base économique explicite.

Cette couche présente une double valeur métier : elle rapproche le système d'une logique de choix économique, plus utile qu'une simple prédiction de rendement ; et elle permet à l'utilisateur d'explorer différents scénarios de prix et de coûts, particulièrement utile lorsque les conditions de marché sont volatiles ou que les hypothèses de coûts varient selon le contexte local.

11. Initialisation des hypothèses économiques

Le script init_economics_db.py initialise la base avec un socle de prix et de coûts par défaut. Les prix initiaux couvrent notamment Maize, Rice, Soybean, Wheat, Cotton, Sorghum et Plantains And Others, avec des références renseignées et des dates d'observation datant de février 2026. Les coûts par défaut sont créés pour douze cultures, mais initialisés à zéro avec la mention explicite « Placeholder values to be edited by the user ».

Cette stratégie est délibérément prudente : les prix de référence rendent l'outil immédiatement démontrable, tandis que les coûts restent volontairement ouverts pour éviter de donner une illusion de précision économique non justifiée. La valeur du système vient de la possibilité d'éditer les hypothèses plutôt que de figer des références prétendument universelles.

12. Interface utilisateur et expérience d'usage

L'interface Streamlit est structurée autour d'un panneau latéral de configuration et de trois onglets principaux : Prédiction, Recommandation et Données économiques. Avant tout usage métier, l'interface interroge /health pour vérifier l'accessibilité de l'API et afficher l'état de chargement des trois modèles.

12.1 Onglet Prédiction

L'utilisateur choisit une culture, renseigne les variables d'entrée, lance la prédiction et obtient le rendement, la marge brute, un tableau de détails, un éventuel avertissement, les entrées réellement utilisées et la réponse JSON brute. L'interface conserve une dimension semi-technique, adaptée à un usage démonstratif ou expert.

12.2 Onglet Recommandation

L'utilisateur demande un classement de cultures, visualise un tableau comparatif, visualise un graphique comparatif (barplot), identifie la meilleure recommandation, lit sa justification textuelle et consulte la liste des cultures écartées faute de données économiques actives.

12.3 Onglet Données économiques

L'utilisateur consulte les prix et coûts actifs, puis peut ajouter ou modifier des valeurs. L'interface signale explicitement que les cultures sans prix actif ne seront pas recommandées tant qu'un prix n'aura pas été saisi, renforçant la cohérence fonctionnelle du produit.

13. Validation des entrées et fiabilité des sorties

L'usage de Pydantic structure fortement la fiabilité des échanges API. Les requêtes PredictRequest et RecommendRequest encadrent les bornes métier minimales : surface strictement positive, pluviométrie non négative, top_k borné entre 1 et 20. Les réponses PredictResponse et RecommendationItem imposent que la borne basse soit inférieure ou égale à la borne haute. Les sorties exposées à l'utilisateur sont typées, cohérentes et documentées.

Au-delà du typage, les réponses incluent la source du modèle utilisé (model_1_crop, model_1_general ou model_2_recommandation), un niveau de confiance, des avertissements, des raisons métier dans les recommandations et un objet inputs_used décrivant les valeurs réellement exploitées avec l'origine des variables climatiques. Cela constitue un premier niveau d'explicabilité applicative pertinent.

14. Industrialisation, qualité et déploiement

Le projet dispose d'un socle d'industrialisation simple mais réel. Le workflow CI s'exécute sur push et pull_request vers main ou master. Il installe Python 3.13, configure uv, synchronise les dépendances avec uv sync --frozen, exécute Ruff en mode vérification et format check, lance Pytest, puis construit les deux images Docker correspondant à l'API et à l'interface.

Le workflow CD publie l'image Docker de l'API vers Docker Hub lors d'un push sur la branche par défaut ou via workflow_dispatch. Il gère Git LFS, configure Buildx, se connecte au registre, vérifie la présence des artefacts et pousse l'image avec un tag latest et un tag court basé sur le SHA du commit.

Le README documente deux modes d'exécution : en local avec uv, et en conteneurs avec docker compose up --build. L'interface est accessible sur le port 8501, l'API sur le port 8000, avec l'accès à la documentation Swagger via /docs.

15. Tests et stratégie de validation technique

La suite de tests couvre le socle API avec trois tests : /health, /predict et /recommend. Les tests utilisent TestClient de FastAPI et des monkeypatch pour injecter des réponses contrôlées aux fonctions métier, ce qui permet de valider les contrats d'API sans dépendre des modèles réels à l'exécution.

Cette couverture vérifie l'essentiel : accessibilité des endpoints, format général des réponses et présence des champs attendus. Elle s'inscrit de façon cohérente dans la CI, où les tests sont exécutés automatiquement après les vérifications Ruff.

16. Valeur métier de la solution

La principale valeur métier est de combiner lecture agronomique et lecture économique dans un même outil. Une prédiction de rendement isolée reste difficile à mobiliser en contexte décisionnel. En ajoutant la surface, les prix, les coûts et un mécanisme de recommandation, l'application se rapproche d'un simulateur d'aide au choix.

Une seconde valeur réside dans la coexistence de plusieurs modèles complémentaires. Un modèle principal culture-spécifique, un modèle général de repli et un modèle secondaire permettent de maintenir une continuité de service malgré l'hétérogénéité des données. Mieux vaut un système capable d'exprimer ses limites et d'adapter sa stratégie qu'un système rigide incapable de traiter certains cas.

Enfin, la possibilité de modifier manuellement prix et coûts donne au projet une valeur pédagogique forte : l'application peut être utilisée pour tester des hypothèses, comparer des scénarios et comprendre comment une décision de culture dépend non seulement du rendement, mais aussi des conditions économiques retenues.

17. Limites de la V1

La V1 présente plusieurs limites assumées. La qualité des recommandations dépend directement de la qualité des prix et coûts actifs saisis dans SQLite : si les données économiques sont absentes, incomplètes ou peu réalistes, la recommandation est

mécaniquement limitée. Le système le reconnaît en signalant explicitement les cultures sans données économiques actives.

La couche économique reste par ailleurs simplifiée. Les coûts par défaut sont initialisés à zéro pour de nombreuses cultures — pratique pour la démonstration, mais insuffisant pour un usage métier avancé. La gestion des unités et devises reste minimale, limitée par exemple à la conversion `usd_per_kg` vers `usd_per_tonne`.

Enfin, les niveaux de confiance et les avertissements relèvent d'une heuristique applicative plutôt que d'une modélisation probabiliste avancée. Ils sont néanmoins utiles pour informer l'utilisateur sur la qualité relative du cas traité.

18. Pistes d'amélioration

Trois familles d'améliorations sont identifiées :

- **Couche économique** : enrichissement des prix et coûts, historisation, meilleure gestion des devises et des unités, raffinement des hypothèses.
- **Interface** : édition avancée des hypothèses économiques, visualisation des cultures actives/inactives, monitoring.
- **Qualité technique** : montée en couverture de tests, validation fine de la couche économique, déploiement plus industrialisé, base de données SQL pour drift du modèle.

19. Conclusion

Le projet 12 aboutit à une application cohérente de prédiction de rendement et de recommandation de culture articulée autour d'une logique métier claire : partir d'entrées agronomiques simples, mobiliser le modèle le plus pertinent selon la couverture de la culture, transformer la prédiction en indicateurs économiques, puis restituer une décision lisible à l'utilisateur.

L'existence d'une interface Streamlit, d'une API FastAPI, d'une base SQLite modifiable et d'une chaîne d'industrialisation minimale. Le projet est donc un prototype permettant d'envisager un passage vers une industrialisation.